

CampusRide — Real-Time Campus Mobility Platform

Design Document · Full-Stack Ride Management System for Campus Transportation

React 19 + TypeScript

Node.js + Express

Socket.IO

PostgreSQL + Prisma

Leaflet / OSM

1. Problem Understanding

Large campuses such as IIT Roorkee rely heavily on e-rickshaws for last-mile transport, yet ride requests and driver availability are coordinated through fragmented, informal channels. This causes idle drivers, long passenger waits, and no visibility into demand. **CampusRide** centralizes this coordination into a single real-time digital platform that connects passengers and drivers, mirroring the core engineering challenges of commercial ride-hailing systems at campus scale.

CORE GOALS

- Secure authentication for passengers and drivers, with driver onboarding and verification.
- A reliable ride lifecycle where a request is assigned to exactly one driver.
- Live, bidirectional updates so both parties always see consistent state.
- Driver insight through dashboards, analytics, and earnings.

KEY CONSTRAINTS

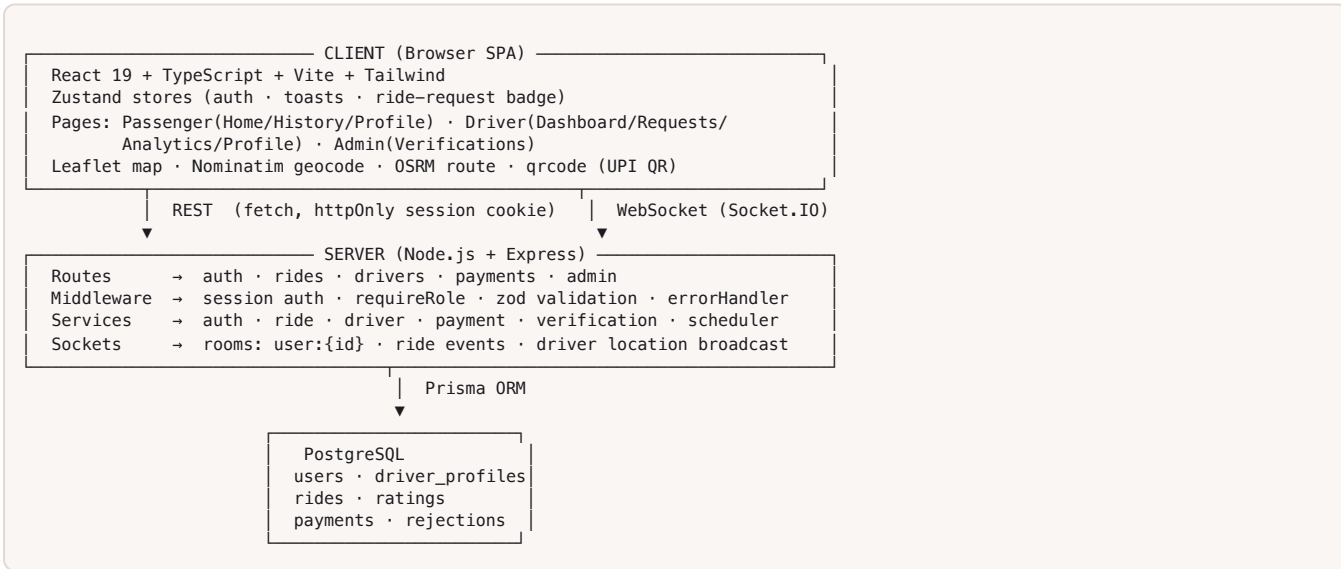
- Consistency:** no double-assignment of a ride under concurrent accepts.
- Responsiveness:** updates must feel instant (WebSocket, not polling).
- Zero-cost geo:** maps and routing without paid API keys.
- Maintainability:** typed end-to-end, modular service layer.

PRIMARY ACTORS

Actor	Capabilities
Passenger	Register/login, set pickup & destination on a map, request or schedule a ride, track the driver live, pay (UPI/QR/Cash), rate & review.
Driver	Onboard with vehicle & verification documents, go online/offline, receive & accept/reject requests, run the ride lifecycle, view earnings & analytics.
Admin	Review pending driver verifications and approve or reject onboarding.

2. System Architecture

CampusRide uses a classic three-tier architecture with a real-time WebSocket channel running alongside the stateless REST API. The frontend is a single-page application; the backend is a layered Express service; PostgreSQL is the single source of truth.



REAL-TIME LAYER

On connection, each socket joins a personal room `user:{id}`. The backend emits targeted events instead of broadcasting globally:

- `ride:requested` → all online drivers
- `ride:accepted` / `ride:status:update` / `ride:cancelled` → the affected passenger & driver
- `driver:location` → the matched passenger only (live marker)
- `driver:status:update` → availability changes

REQUEST FLOW EXAMPLE

Accept a ride:

- Driver taps Accept → `PUT /rides/:id/accept`
- Service runs a conditional `updateMany` guarded on `status = REQUESTED` — only the first writer succeeds.
- On success, server emits `ride:accepted` to both rooms.
- Passenger UI shows driver details; driver is routed to the dashboard to start the ride.

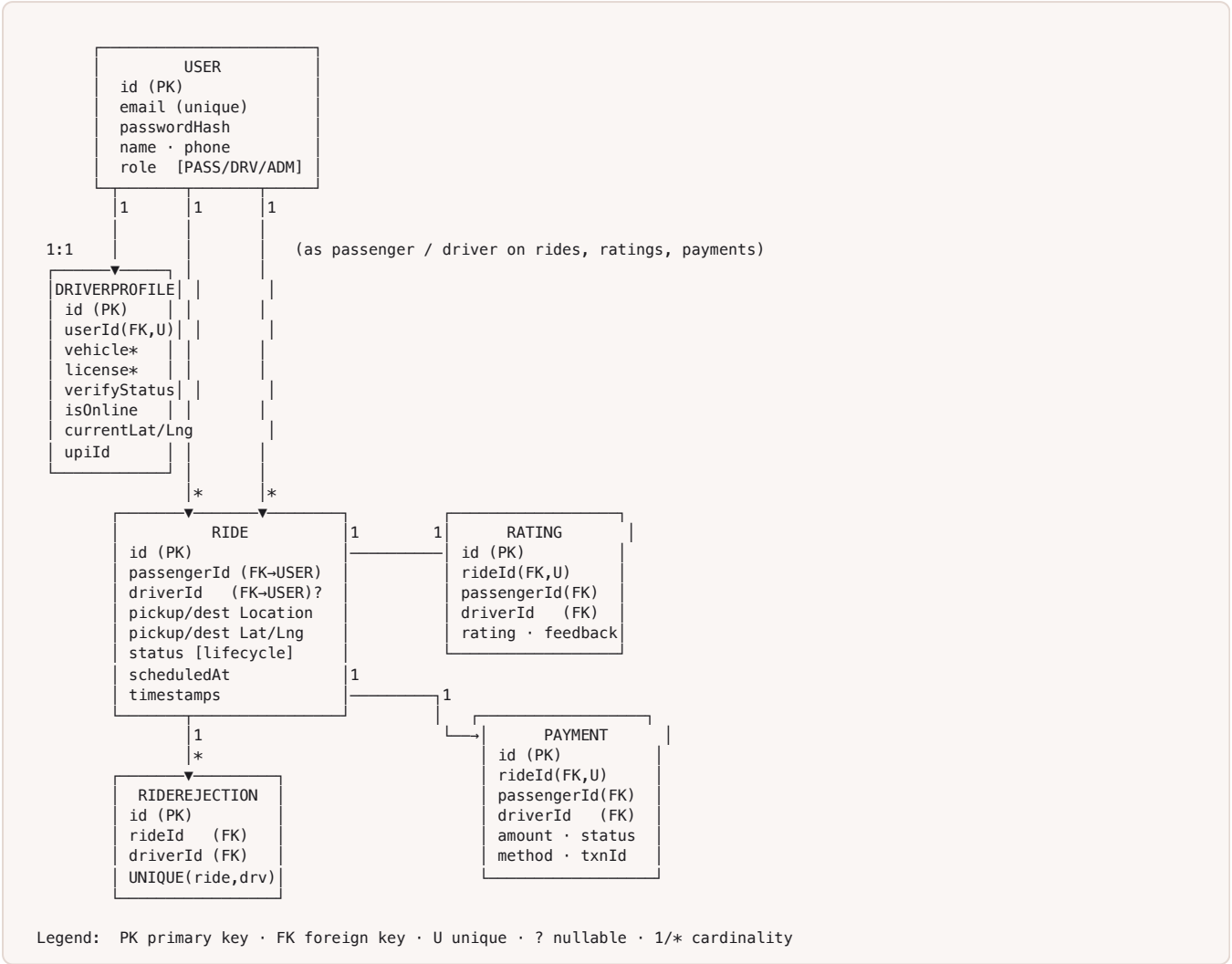
3. Database Schema

Modeled with Prisma over PostgreSQL. All tables use CUID primary keys and snake_case columns. Strategic composite indexes back the hot query paths (pending requests, a driver's active ride, a passenger's history).

Model	Key Fields	Notes
User	id, email (unique), passwordHash, name, phone, role	role ∈ {PASSENGER, DRIVER, ADMIN}. bcrypt-hashed password.
DriverProfile	userId (unique), vehicleType, vehicleNumber, licenseNumber, governmentIdNumber, verificationStatus, isOnline, currentLat/Lng, upild	1-to-1 with User. Holds onboarding, verification, live location & payment handle.
Ride	passengerId, driverId?, pickup/destLocation, pickup/destLat/Lng, status, scheduledAt, cancelReason, timestamps	Central lifecycle entity. status ∈ {REQUESTED, ACCEPTED, IN_PROGRESS, COMPLETED, CANCELLED}.
Rating	rideId (unique), passengerId, driverId, rating, feedback	One rating per completed ride; aggregates into driver average.
Payment	rideId (unique), passengerId, driverId, amount, status, method, upiTransactionId	Auto-created on ride completion. method ∈ {UPI, QR_CODE, CASH}.
RideRejection	rideId, driverId, rejectedAt	Unique (rideId, driverId) — hides a request from a driver who rejected it.

Indexing strategy. rides carries composite indexes on (status, requestedAt), (status, scheduledAt), (driverId, status) and (passengerId, status); payments on (passengerId, status) and (driverId, status). These keep the request feed, scheduler poll, active-ride lookup, and history queries index-backed rather than full scans.

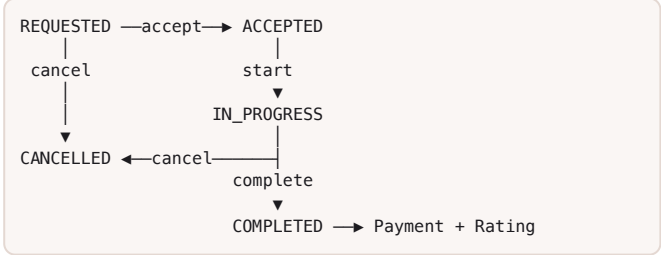
4. Entity Relationship Diagram (ERD)



RELATIONSHIPS

- **User 1:1 DriverProfile** — only driver accounts have a profile.
- **User 1:* Ride** — twice, as passenger and as driver.
- **Ride 1:1 Rating** — at most one rating per ride.
- **Ride 1:1 Payment** — created automatically on completion.
- **Ride 1:* RideRejection** — tracks per-driver dismissals.

LIFECYCLE STATE MACHINE



5. API Overview

REST over JSON. Auth via httpOnly session cookie; role-guarded routes use `requireRole`. All bodies validated with Zod.

AUTH /API/AUTH

POST	/register
POST	/login
POST	/logout
GET	/me
PUT	/profile

RIDES /API/RIDES

POST	/ request / schedule
GET	/active · /history · /pending

DRIVERS /API/DRIVERS

GET	/available · /dashboard · /analytics
PUT	/status online/offline
PUT	/location live GPS
PUT	/vehicle · /verification · /upi

PAYMENTS /API/PAYMENTS

GET	/ride/:rideId · /history
PUT	/:id/complete UPI/QR/Cash

ADMIN /API/ADMIN

PUT	/:id/accept · /reject
PUT	/:id/start · /complete · /cancel
POST	/:id/rate

GET	/verifications/pending
PUT	/verifications/:userId

WEBSOCKET EVENTS

`ride:requested` · `ride:accepted` · `ride:status:update` ·
`ride:cancelled` · `ride:rejected` · `driver:status:update` ·
`driver:location`

6. Design Decisions

Decision	Choice & Rationale
Single-driver assignment	Accept is a conditional <code>updateMany</code> filtered on <code>status = REQUESTED</code> . The database enforces the race-condition guard atomically, so concurrent accepts can never double-assign — no application-level locking needed.
Real-time via Socket.IO rooms	Each user joins <code>user:{id}</code> at connect time. Events are emitted to specific rooms rather than broadcast, keeping traffic targeted and avoiding any client-side <code>ride:join</code> handshake for location updates.
Session cookies over JWT	HttpOnly, secure cookies backed by a server session store resist XSS token theft and allow instant revocation on logout — simpler and safer than managing JWT refresh rotation for a campus-trust app.
Layered service architecture	Routes handle HTTP + validation only; all business logic lives in framework-agnostic services. This keeps logic testable and reusable across REST and socket handlers.
Zero-key geospatial stack	Leaflet + OpenStreetMap tiles, Nominatim geocoding, and OSRM routing are all free and keyless — fully reproducible by judges with no secret provisioning. Driver GPS uses the browser <code>watchPosition</code> API, pushed every 10s.
Real UPI payments, app-recorded	QR codes encode a standard <code>upi://pay</code> intent, so scanning in any UPI app triggers a genuine transfer to the driver. The platform records the payment and method without a paid gateway, keeping the flow real yet self-contained.
Scheduled rides via poller	A lightweight interval poller promotes due scheduled rides into the live request feed, reusing the same notification path as instant rides — no separate dispatch logic.
End-to-end TypeScript	Shared types across client and server with Prisma-generated models eliminate a whole class of integration bugs and make refactors safe.

Scalability outlook. The stateless REST tier scales horizontally behind a load balancer; Socket.IO can scale across instances with a Redis adapter to share rooms. PostgreSQL read replicas can absorb analytics/history load, while the indexed hot paths keep dispatch latency low as ride volume grows.